LiDAR × 模倣学習による TurtleBot3 自律走行

Physical AI 応用1講座 最終課題 説明資料 ／ ROS 2 Humble・Gazebo Classic 11・TurtleBot3 Burger・PyTorch
コード: https://github.com/2life2021/tb3-ai-navigator ／ 動画: YouTube

1. 概要と目的

TurtleBot3 が LiDAR の距離情報 `/scan` だけを使い、**ニューラルネットワークの一種である多層パーセプトロン（MLP）の推論結果**で「前進・左旋回・右旋回」を選び、`/cmd_vel` に速度指令を出して、障害物のある環境 turtlebot3_world を衝突せずに走り続けることをタスクとしました。

if文で直接ロボットを動かすのではなく、**ルールで書いた「教師」の振る舞いを、シミュレーション内で自動収集したデータから模倣学習**させ、走行中は学習済みモデルだけで判断させます。さらに、**ロボットを実際に走らせて定量評価し、失敗を分析して改善するサイクル**を回したことが本作品の中心です。

実世界では、掃除ロボットや倉庫の搬送ロボットの「ぶつからずに動き続ける」という基本機能に相当します。ルールを人手で調整し続けるかわりに、データからセンサ→行動の対応を学ぶ構成にしておくと、人間の操作ログや強化学習など、教師を別のものに差し替えて発展させることができます。

2. システム構成

```
Gazebo（turtlebot3_world）
  ｜ /scan  LaserScan 360本，約5Hz
  ▼
ai_driver.py（ROS 2 ノード）
  前処理：inf→3.5m，5°ごと72セクタの最小距離，÷3.5 で 0〜1 に正規化
  MLP：72 → 128 → 128 → 3（ReLU，softmax）  ……  P(前進)，P(左)，P(右)
  制御：argmax = 最大確率の行動  ／  soft = 確率で速度を重み付け平均
  ｜ /cmd_vel  Twist   前進 v=0.18m/s ／ 左右 v=0.02m/s，ω=±0.9rad/s
  ▼
Gazebo 上の TurtleBot3 Burger            ai_monitor.py：LiDARと確率を可視化
```

3. AIモデル：入力・出力・学習方法

項目	内容
モデル	多層パーセプトロン。入力72 → 隠れ層128×2 → 出力3。活性化関数はReLU、実装はPyTorch
入力	LiDAR 360本を5°ごとの72セクタに分け、各セクタの最小距離を0〜1に正規化したベクトル
出力	前進・左旋回・右旋回の3行動の確率
学習方法	模倣学習。教師あり3クラス分類として、クラス重み付き交差エントロピーとAdamで学習
学習データ	Gazebo で自動走行しながら収集した、生LiDAR 360本と行動ラベルの組3,206件。学習用の2,565件を、左右反転とノイズ付加で25,650件に拡張
正解ラベル	**ルールベースの教師 teacher_action_v2 で自動生成。**前方±30°が0.55m以上、斜め前30〜75°が0.35m以上空いていれば前進。ただし、真横75〜105°に0.22m未満の壁があれば、その壁から離れる方向へ旋回。それ以外は、左右20〜100°で平均距離が大きい方へ旋回
推論	`/scan` を受信するたびに、CPUで約5Hzで推論。走行中の行動は MLP の出力だけで決める

> **AIの役割と限界**
>
> - 走行中の判断は MLP のみが行います。教師ルールは、評価指標である教師一致率を計算するためだけに呼び出しています。
> - 正解ラベルがルール由来なので、性能の上限は基本的に教師と同程度です。「AIが教師より賢くなる」ことは主張しません。
> - 地図や自己位置推定を持たない反応型の方策なので、ゴールへの移動や探索はできません。学習した環境以外での性能は検証していません。

4. 開発の経緯と工夫した点

① まず最小構成を動かす：v1

教師v1でデータ1,176件を収集し、15°刻み24セクタの入力でMLPを学習しました。検証精度は91%でした。120秒の走行で衝突0回、18.95m を走行し、合格ラインの動作を確認しました。

② DAgger を試したら悪化 → ログを解析して原因の仮説を立てる

模倣学習には、AIが自分で操作すると学習データにない状態に入ってしまう「共変量シフト」という弱点があります。その対策として、AI自身が走ったときの状態に教師ラベルを付けて追加学習する DAgger を導入しました。ところが、検証精度は91.1%から93.9%に上がったのに、初期位置をそろえた120秒の走行で**衝突イベントが0回から6回に増えました。**比較は各条件1回ずつです。

観測事実: 6回のうち4回は左下の柱付近、2回は外周の壁付近で起きました。直前のログには、前進確率≈1.0のまま最小距離が縮んでいく場面と、その場旋回中に最小距離が縮む場面がありました。また、教師v1は前方±60°しか見ておらず、真横の障害物を判断に使っていません。
仮説: 教師v1が「横の障害物に近い状態での前進」を正解としていたため、DAggerで障害物近くの状態データが増えたことで、その判断が強化された。ただし、ログには最小距離の方向を記録していないためこの仮説は直接確かめられず、1回ずつの比較なので偶然の可能性も残ります。
この分析から「モデルよりも教師ラベルの質を疑うべき」と判断し、真横の確認を加えた教師v2を作りました。

③ 生データを保存して再ラベル付け

収集時に生LiDAR値も保存していたので、教師を改良しても**再走行せずに全データへラベルを付け直せます。**

④ 特徴量の角度分解能とデータ拡張

15°刻みのセクタでは、教師が使う角度の境界である30°・75°・105°と、セクタの境界がずれるため、5°刻み72セクタに変更しました。さらに、左右反転とσ=2cmの距離ノイズを加えたスキャンを教師で再ラベルして、学習データを約10倍に拡張しました。検証には、拡張していない元データを使いました。

⑤ 確率を使った soft 制御と可視化

最大確率の行動を選ぶ argmax 方式に加えて、行動確率で速度を重み付け平均する soft 制御も実装しました。確率が割れていて確信度が低いときは自動的に減速し、動きも滑らかになります。また、AIが見ているLiDARと出力確率をリアルタイムに表示するモニターを作り、動画で「AIが何を見て、どう判断したか」がわかるようにしました。

5. 評価

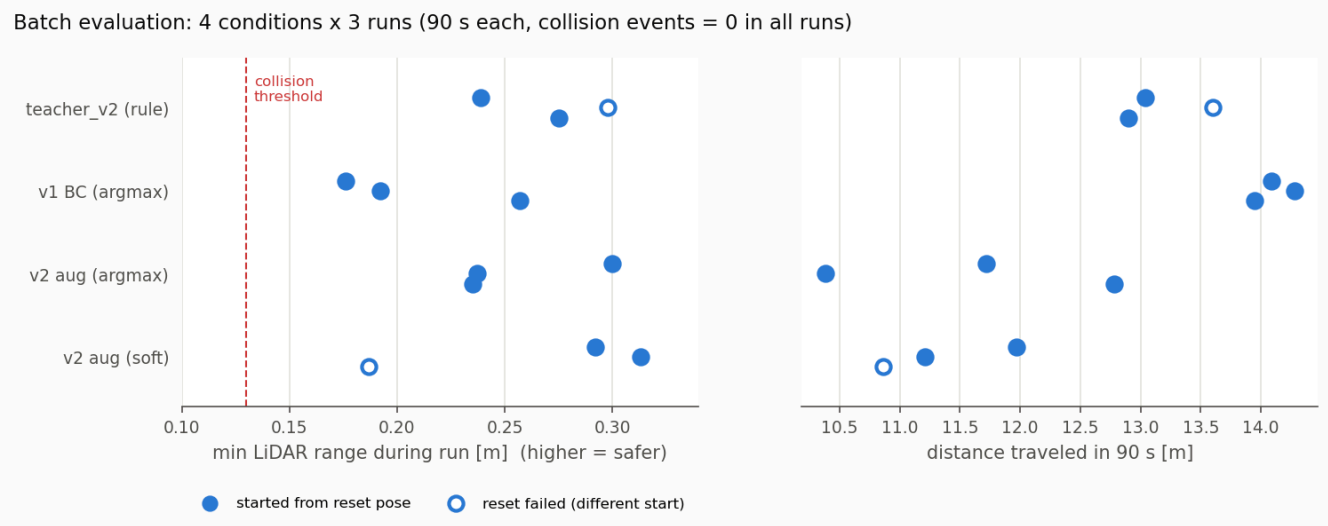
各試行の開始時に `/reset_world` を呼んで初期位置に戻し、各条件を複数回走らせて自動記録しました。ただし、12回中2回はリセットに失敗しました。

評価指標: 走行距離は /odom の積算値。最接近距離は走行中の LiDAR 最小値で、大きいほど安全。衝突イベントは LiDAR 最小距離が0.13m未満になった回数で、接触センサによる判定ではない。教師一致率は、AIの行動が教師ルールと一致した割合。

一括評価：4条件 × 3回 × 90秒

条件	試行	走行距離 [m]	最接近距離 [m] 平均 / 最小	衝突イベント	教師v2との一致率
教師v2のルールで走行	3	13.2 ± 0.3	0.271 / 0.239	0	1.00
v1 BC argmax	3	14.1 ± 0.1	0.208 / 0.176	0	0.79 ※1
v2 aug argmax［最終採用］	3	11.6 ± 1.0	0.257 / 0.235	0	0.80
v2 aug soft	3	11.3 ± 0.5	0.264 / 0.187	0	0.83

※1 v1は教師v1で学習したモデルなので、教師v2との一致率は参考値です。※2 教師v2の2回目とsoftの1回目は `/reset_world` が効かず、別の位置から始まっています。図では白抜きの点で示しています。初期位置が正しかった回だけで集計すると、最接近距離は教師v2が平均0.257m・最小0.239m、softが平均0.303m・最小0.292mです。



全12回とも衝突イベントは0回でした。最終採用モデルは、v1に比べて障害物との距離を大きく保ち、最接近距離の最小値は0.176mから0.235mに増えました。その代わり旋回が増えて、走行距離は14.1mから11.6mに減りました。事後分析では、全試行で0.25m四方のマスを36〜64個訪れており、5秒間で0.10m未満しか動かない立ち往生は最大4.4秒でした。

デモ動画の走行: 最終採用モデルで初期位置から150秒走行させて撮影しました。走行距離17.8m、衝突イベント0回、最接近距離は **0.187m** でした。一括評価での最小値0.235mより障害物に近づいており、3回の評価だけでは最悪ケースを把握しきれていないことを示しています。

実験設計の監査

項目	確認結果
初期位置・向き	12回中10回は同じ位置から開始し、2回はリセットに失敗した。**コードがリセットの成否を確認していなかった**。向きはログに記録しておらず、検証できない

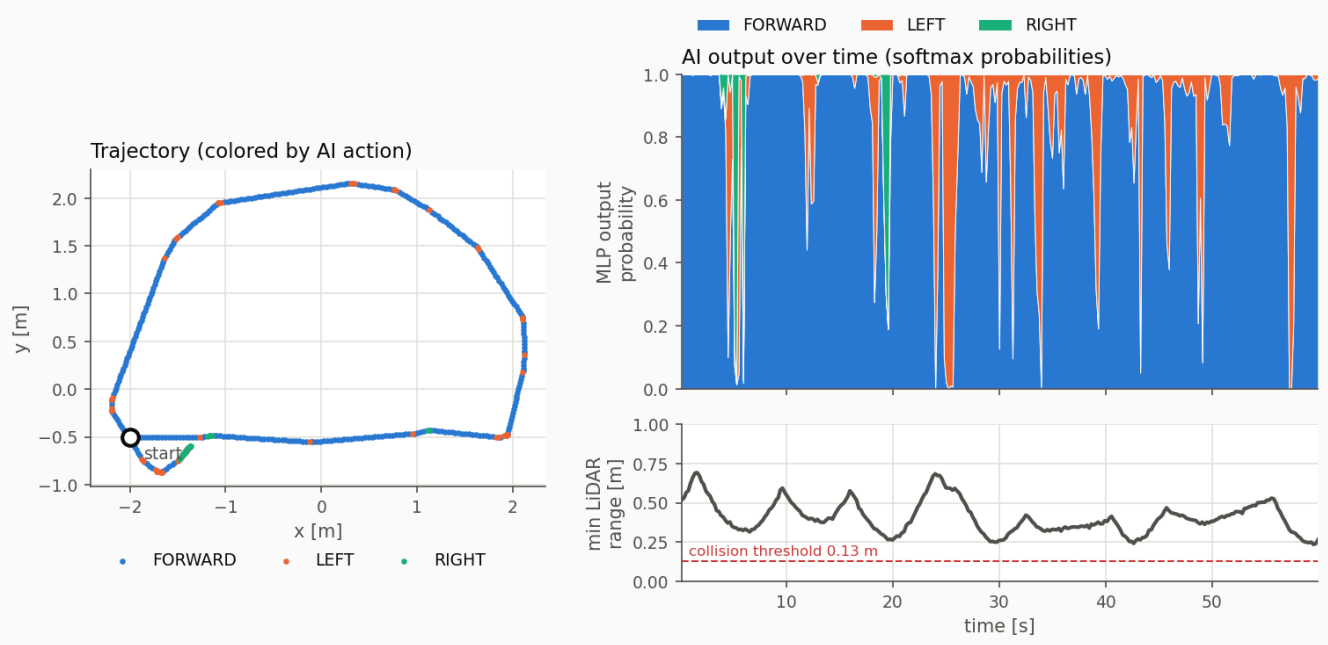
試行の多様性	同じ初期位置で3回なので、違いはσ=1cmのLiDARノイズとタイミングの揺らぎだけ。初期位置への依存性は評価できておらず、統計的な有意差も言えない
衝突の定義	LiDAR最小距離が0.13m未満になった回数。接触センサではないため、LiDARの死角での接触は検出できない
評価コードと提出コード	評価の後に追加したのは可視化用トピックの送信だけで、行動の決定と速度指令の計算は同一。提出コードでは短い動作確認だけを行った
改善案	リセット後の位置確認と再試行、yawの記録、複数の初期位置×5回以上、接触センサによる判定、提出コードでの再評価

学習の検証精度

モデル	教師	特徴量	データ	検証精度
v1	v1	24セクタ	1,176	91.1%
v1 + DAgger	v1	24セクタ	2,039	93.9%
v2 24セクタ	v2	24セクタ	3,206・再ラベル	87.1%
v2 72セクタ	v2	72セクタ	3,206・再ラベル	88.3%
v2 aug	v2	72セクタ	3,206・再ラベル＋拡張	87.2%

v1とv2では教師が異なるため、精度を直接比較することはできません。検証精度が最も高い「v1+DAgger」が走行では最も悪かったことから、**オフラインの精度だけでは走行性能を判断できず、閉ループ評価が不可欠**だとわかります。

最終採用モデルの走行例



左は軌跡で、点の色はAIが選んだ行動を表します。右上は最初の60秒間にMLPが出力した各行動の確率、右下はLiDARの最小距離です。

考察

- **模倣学習の質は教師ラベルの質に強く依存する**: DAggerの悪化は、手法そのものより、真横を見ていないという教師ルールの欠陥による可能性が高いと考えています。ただし、障害物の方向を記録していないためこれは未検証の仮説です。それでも、閉ループのログ解析がなければ、この仮説にたどり着くこともできませんでした。
- **検証精度は約87%で頭打ち**: 誤りの多くは前進と旋回の閾値付近で起きており、ラベル自体があいまいになる状態です。

6. 再現性と開発の記録

詳細は GitHub の `docs/REPRODUCIBILITY.md` と `docs/DEVELOPMENT_LOG.md` にあります。

何をAIが学習し、何を人間が決めたか

本作品で**AIが学習によって決めたのは、MLPの重みとバイアスだけ**です。それ以外の設定は、すべて人間が決めました。

AIが学習で決めたもの

決めたもの	役割	決まり方
MLPの重みとバイアス 26,243個	LiDARの特徴量から、前進・左旋回・右旋回それぞれの確率を計算する式の係数	教師ラベルとの誤差が小さくなるように、誤差逆伝播とAdamで自動的に調整される

人間が決めたもの：開発の流れの順

段階	決めたもの	値	何に影響するか
① タスクの設計	行動の種類と速度	前進 0.18m/s ／ 左右旋回 0.9rad/s	ロボットが取れる動きそのもの
② 正解の定義	教師ルールの閾値	v2: 前方±30°が0.55m以上、斜め前30〜75°が0.35m以上なら前進。真横75〜105°が0.22m未満なら反対側へ旋回	**AIが「正しい」と学ぶ行動の基準**。AIの振る舞いを最も強く左右する
③ 入力の作り方	LiDARのまとめ方	5°ごとの72区間の最小距離を、3.5で割って0〜1にそろえる	AIが周囲をどの細かさで見るか
④ データの集め方	ランダム行動の確率 ε	0.10、DAggerの回のみ0.05	危ない状態のデータがどれだけ含まれるか
	DAggerで教師が操作する確率 β	0.3	AI自身が運転した状態をどれだけ集めるか
	収集時間	240秒・180秒・240秒の3回、計3,206件	データの量
⑤ 学習の設定	モデルの大きさ	隠れ層 128 × 2層	表現できる判断の複雑さ
	学習の回数・進め方	エポック150、バッチ128、Adam 学習率0.001	学習がどこまで、どれだけ速く進むか
	データ拡張	左右反転＋ノイズで学習データを10倍	未知の状態への強さ
	検証データの割合と乱数シード	20%・シード0	精度の測り方と、学習の再現性
⑥ 評価のルール	衝突イベントの基準	LiDAR最小距離 0.13m未満	何を「衝突」と数えるか
	試行時間と回数	90秒 × 3回	評価の信頼性

表の値は最終採用モデルのものです。各モデルの値の違いは GitHub の `docs/REPRODUCIBILITY.md` にまとめています。

データ収集時の教師の条件

データ	運転していたもの	保存ラベル	ε / β	時間	件数	前/左/右
teacher	教師v1	v1	0.10 / –	240s	1,176	1020/85/71
dagger1	v1モデル。確率0.3で教師v1	v1	0.05 / 0.3	180s	863	721/84/58
teacher_v2	教師v2	v2	0.10 / –	240s	1,167	717/233/217

v2系のモデルは、3つのデータすべてを保存済みの生LiDARから**教師v2で再ラベル**して学習しました。つまり、**最終モデルの学習データには、DAggerで集めた状態863件が教師v2のラベルで含まれています**。悪化したのは「v1のラベルのまま追加学習した」条件であり、DAggerの状態データが最終モデルにどれだけ寄与したかは、切り分けて評価していません。

推論・制御の変更で起きた問題と修正

問題	原因	修正 → 結果
開始位置が試行ごとに違い、比較が不公平だった	確認済み: 各試行が前回の終了位置から始まっていた	`--reset` を追加 → 以後はほぼ同じ位置から開始。12回中2回は失敗
`--reset` を付けると AttributeError で落ちた	確認済み: 初期化の途中でサービスの応答を待ったため、scan のコールバックが先に実行された	リセット処理を購読の登録より前に移した → 解消
v1+DAggerで衝突イベント6回。検証精度は最高だった	推定: 教師v1が真横を見ず、障害物の近くでの前進を正解としていた	教師v2を作成 → 教師単体の最接近距離 0.198→0.234m
最初のv2モデルの検証精度が79%	推定: 約1,100件というデータ不足と、モデルの幅不足	全データを再ラベルし、幅128に → 87.1%
120秒の指定で290秒かかり、scanが約0.9Hzに低下	未解明。直後の計測では4.86Hzで正常	予防策として推論を1スレッドに → 以後は再発なし
入力の次元がモデルによって24と72で異なる	設計上の問題: 特徴量を呼び出し側で計算していた	セクタ数をモデルに保存し、推論側で特徴量を計算 → 次元の取り違えを防止
argmaxでは迷っていても全速で動く	制御方式の性質	soft制御を追加 → 衝突0回で同程度。優劣は判断できず

再現性の評価

- **学習は再現できる**: 同じデータ・設定・シードで別々に学習した2つのモデルで、学習精度0.98441・検証精度0.87051が完全に一致しました。
- **データ収集と走行は完全には再現できない**: 乱数シードを固定しておらず、LiDARノイズや通信のタイミングでも軌跡が変わります。収集時の開始位置と、すべての試行のyawは記録していません。時間はシミュレーション時間ではなく実時間で区切っています。
- **記録から得た学び**: 検証精度93.9%で最も高かったv1+DAggerが走行では最も悪く、データ拡張は精度を上げなかったのに走行では障害物から距離を保ちました。オフラインの指標と閉ループの性能は別物です。また、開始位置、一致率の基準、シミュレーション速度など、気づかないうちに比較条件が変わる要因が多くありました。どれも、ログを残していたから後で気づけたものです。

7. 制作を通して得た学びと今後の課題

今回の課題では、ROS 2と機械学習を利用したTurtleBot3の自律走行システムを制作しました。その過程で、開発環境の構造、ロボットの通信・制御、ニューラルネットワークの学習原理について理解を深めました。

7.1 開発環境の階層構造

最も大きな学びは、開発環境を構成する各技術の役割と階層関係でした。

OSはコンピュータ全体の動作を管理し、その中核である**カーネル**がプロセスやメモリ、ハードウェア資源などを管理していることを学びました。一方、PowerShellやBashなどの**シェル**は、ユーザーがコマンドを通じてOSやプログラムを操作するためのインターフェースであると理解しました。

また、Dockerはコンテナによって実行環境を管理する仕組みであり、コンテナは原則として独立したカーネルを持たず、ホスト側から提供されるカーネルを利用することを学びました。今回のWindows上で動作するLinuxコンテナも、Linuxカーネルを利用できる仮想化環境を介して実行されていると理解しました。

さらに、**ROS 2はノード間の通信を担うミドルウェア、Gazeboはロボットの運動を再現する物理シミュレータ**であり、これらはOSやシェルとは異なる階層に分類されることを学びました。

このように、開発環境を単なるソフトウェアの集合ではなく、異なる役割を持つ技術が階層的に連携するシステムとして理解できました。

7.2 ROS 2の通信構造

ROS 2では、**ノードが処理を担当する単位であり、トピックはノード間で情報を交換するための通信経路**であることを理解しました。例えば、LiDARの観測情報は `/scan`、ロボットへの速度指令は `/cmd_vel` というトピックを通して送受信されます。

また、Pythonの `self.on_scan` と `self.on_scan()` の違いも重要だと学びました。前者はメソッドそのものを参照し、後者はメソッドを呼び出します。この違いから、センサー情報が届いた際に、事前に登録した処理が実行される**コールバック**の仕組みを理解できました。

さらに、`/odom` はロボットの位置や姿勢に関する推定情報を提供しますが、その値には誤差が含まれる可能性があり、正確な自己位置を保証するものではないことを学びました。オドメトリは車輪の回転量などから求めた移動量を積み重ねて位置を推定するため、1回ごとのわずかな誤差が時間とともに**少しずつ蓄積していき**、走行が長くなるほど実際の位置とのずれが大きくなります。これが、オドメトリの誤差が問題になる理由だと理解しました。

特に、**オドメトリによる位置推定と、LiDARによる障害物の認識は異なる機能**であると区別することが重要だと理解しました。

7.3 ロボットの認識と制御

今回のAIでは、LiDARから取得した360方向の距離情報を5度ごとの72区間に分割し、各区間の最短距離を抽出することで、72次元の特徴量に圧縮しました。360を72で割ると、1区間はちょうど5度になります。

この処理により、入力する情報量を削減しながら、各方向で最も近い障害物の情報を残せることを学びました。一方で、圧縮によって元の観測情報の一部は失われることも理解しました。

また、今回の**MLPへの入力は `/scan` に由来するLiDAR情報のみであり、`/odom` による位置・姿勢情報は使用していません。**したがって、今回のシステムは目的地までの経路を計画するナビゲーションではなく、現在の周囲状況に応じて前進・左旋回・右旋回を選択し、障害物との衝突を回避しようとする反応型の制御システムです。

ここから、**障害物を回避する能力と、地図や自己位置を利用して目的地に到達する能力は異なる**と理解しました。

7.4 機械学習の構造と学習原理

MLP・ニューロン・パラメータ
今回のMLPは、72次元の入力から128個・128個のニューロンを持つ隠れ層を経て、3種類の行動を出力する 72 → 128 → 128 → 3 の構造です。

ここで、128というニューロン数は人間が設定した**ハイパーパラメータ**であり、各ニューロンの計算に使用される重みとバイアスは、学習によって調整される**モデルパラメータ**であることを学びました。今回のモデルには、モデルパラメータが合計26,243個存在します。

各ニューロンは入力の重み付き和とバイアスを計算し、活性化関数を通して次の層に情報を渡すことを理解しました。

隠れ層で生成される128個の中間特徴量は、必ずしも人間が解釈できる意味や、物理量同士の相関を表しているわけではなく、教師の行動を予測する際の損失が小さくなるように形成された、数学的な中間表現であると理解しました。

また、72個の入力から128個の特徴量を生成しても、元の入力に存在しない独立した情報が増えるわけではなく、既存の情報を異なる形に変換し、より複雑な判断規則を表現できるようにしていることを学びました。

教師データとデータ拡張
今回のAIは、人間が設計したルールベースの教師の行動を模倣する**行動クローニング**を中心として学習しました。

学習データ3,206件は手動で個別に作成したものではなく、Gazebo上でロボットを自動走行させ、LiDAR情報と教師の判断を記録して収集しました。さらに、学習用に分割した2,565件のデータに左右反転や距離ノイズを加え、25,650件に拡張しました。

ここで増やしたのは、MLPのニューロン数ではなく、**教師ラベル付きの学習データ**です。つまり、ニューロン数はモデルの構造を決める設定、重み・バイアスは学習する対象、データ拡張は学習材料を増やす操作であり、それぞれ役割が異なることを理解しました。

損失関数と学習
MLPが出力する3つの値はロジットと呼ばれ、そのまま確率を意味するものではないことを学びました。Softmax関数によって合計1の確率分布に変換できますが、**その値が現実の成功確率として正確である保証はない**ことも理解しました。

今回の学習では、ロジットと教師ラベルから交差エントロピー損失を計算し、誤差逆伝播によって各パラメータの勾配を求め、Adamを用いて重みとバイアスを更新しました。PyTorchの交差エントロピー損失には、Softmaxに相当する処理が内部に含まれていることも学びました。

また、バッチサイズや学習率は人間が設定するハイパーパラメータであり、重み・バイアスとは異なると理解しました。

MLPを利用する意義と評価の限界
今回の教師ルールは、それ自体で障害物回避を実行できます。それでもMLPを用いた意義は、**人間が設計した判断規則を、データからニューラルネットワークに学習させる仕組みを実装・検証すること**にあると考えました。

ただし、MLPが教師より優れているとは限らず、モデルの表現力が高いことと未知環境への汎化性能が高いことも同義ではないと学びました。

今回得られた検証精度87.2%は、**教師ルールとの行動一致率であり、障害物回避の成功率ではない**と理解しました。また、学習データと検証データは同じ収集データから分割しているため、観測内容が似ている可能性があります。したがって、シ

ミュレーションでの成功や教師との一致率だけで、未知の障害物配置や実環境でも適切に動作すると判断することはできないと学びました。

7.5 限界・課題・改善案

今回のシステムでは、Gazebo上でLiDARを利用した障害物回避と、その判断をMLPに模倣させる仕組みを実現しました。一方、実環境への応用や機械学習モデルの有効性を検証するうえでは、以下の限界と課題が残されています。

① 未知環境への汎化性能
限界：今回の評価は主に特定のシミュレーション環境におけるものであり、未知の障害物配置でも同様に動作するとは限りません。
課題：学習データと検証データは同じ走行記録からランダムに分割しているため、互いに類似した観測が含まれる可能性があります。結果として、検証精度が未知環境での性能を十分に反映していないおそれがあります。
改善案：学習用と評価用で障害物の配置や初期位置を明確に分離し、未経験の環境における走行成功率、障害物との最短距離、停止・接触回数などを比較します。

② 教師ルールへの依存
限界：MLPが学習する正解は人間が設計した教師ルールの判断であり、必ずしも最適な行動とは限りません。
課題：教師ルールが不適切な判断をする状況では、その判断をMLPも模倣する可能性があります。また、MLPを導入することで教師より性能が向上したとは確認できていません。
改善案：教師ルール自体を改善するとともに、人間の操縦データや、衝突回避以外の評価基準を利用した学習方法を検討します。また、教師とMLPを同一条件で比較し、MLPを利用する利点を定量的に検証します。

③ 自己位置推定と経路計画の欠如
限界：今回のMLPはLiDAR情報のみを入力としており、自己位置や目的地を考慮した行動選択は行いません。
課題：現在の障害物を回避できても、目的地への到達や効率的な経路選択を保証できません。また、オドメトリには誤差が生じ得るため、単に `/odom` を入力に追加するだけでは正確な自己位置が得られるわけではありません。
改善案：SLAMや自己位置推定、経路計画の仕組みを導入します。さらに、推定した自己位置や目標地点の情報を制御に利用することで、単純な障害物回避から目的地への自律移動へ発展させます。

④ モデル構造と特徴量の妥当性
限界：今回の入力72次元、隠れ層128・128という構造が、最適であるとは確認できていません。
課題：ニューロン数や特徴量数を増やすと表現力は高まる可能性がありますが、学習データへの過適合や計算量の増加につながる場合もあります。また、5度ごとの最短距離への圧縮では、元の距離情報の一部が失われます。
改善案：入力次元を36・72・360、隠れ層の幅を64・128・256などに変更し、検証精度だけでなく、未知環境での走行性能や推論速度も比較します。これにより、モデルの複雑さと性能の関係を検証します。

⑤ 評価指標と実験条件の不足
限界：検証精度87.2%だけでは、実際の障害物回避性能や安全性を十分に評価できません。
課題：教師との行動不一致には、危険な前進と不要な旋回など、影響の異なるものが含まれます。また、今回の走行評価は試行回数が限られ、初期状態のリセットにも失敗した試行があったため、結果の信頼性を十分に検証できていません。
改善案：実験の初期条件と乱数を管理し、試行回数を増やします。衝突の有無、最短障害物距離、走行距離、停止時間などを記録し、平均値だけでなく結果のばらつきも比較します。また、危険な行動の誤分類を重点的に評価します。

⑥ 実機への適用と再現性
限界：今回の動作確認はシミュレーション上で行っており、実機で同じ性能を発揮できるとは限りません。
課題：実機ではセンサーの測定誤差、車輪の滑り、通信遅延、シミュレーションと現実の物理特性の違いなどが生じます。また、第三者が同じ実験を再現するには、コードだけでなく環境構築や実験条件の共有も必要になります。
改善案：センサーノイズや摩擦条件を変えたシミュレーション実験を行い、実環境との差に対する頑健性を確認します。そのうえで、安全対策を設けた実機試験へ段階的に移行します。また、Docker環境、依存ライブラリ、学習条件、実験手順をGitHub上に明示し、第三者による再現性を高めます。

7.6 総括

今回の最大の学びは、ロボットの自律走行を単一のAIの働きとしてではなく、**開発環境・通信・センシング・制御・機械学習が階層的に連携するシステム**として理解できたことです。

また、モデル構造と学習パラメータ、教師データとデータ拡張、障害物回避とナビゲーション、教師との一致率と実際の成功率など、似ている概念を明確に区別する重要性を学びました。

特に、**何を入力し、何を正解として学習し、どのような指標で評価しているのかを正確に把握すること**が、機械学習システムの仕組みと限界を理解するうえで不可欠だと実感しました。

今後は、未知環境における評価の独立性を高めるとともに、モデル構造や教師ルールの改善を検証する必要があると考えています。さらに、自己位置推定や経路計画を導入することで、現在の反応型障害物回避から、より高度な自律移動システムへ発展させることが課題です。

7.7 今後の研究方針

今後は、単にモデルの分類精度を向上させるのではなく、**実際の目的である安全な障害物回避に対して、機械学習がどの程度有効なのか**を検証したいと考えています。

特に、教師ルールとMLPの走行性能を同一条件で比較するとともに、未知の障害物配置における汎化性能を評価します。また、入力次元や隠れ層の構造を変更することで、モデルの複雑さと実際の制御性能の関係を明らかにしたいと考えています。

これにより、機械学習を導入すること自体を目的とするのではなく、**どのような条件で、なぜ機械学習が有効になるのか**を検証することを今後の課題とします。

8. 環境構築と実行方法

層	内容
ホスト	Windows 11。GPUは不使用
WSL2 / Docker	WSL2 上の **Docker Desktop 29.8.2**。コンテナは独自のカーネルを持たず、WSL2のLinuxカーネルを共有する
コンテナ	イメージ `airobotbook/ros2-desktop-ai-robot-book-humble`。Ubuntu 22.04.5、ROS 2 Humble、Gazebo 11.10.2、Python 3.10.12、PyTorch 2.6.0。**本作品のための追加インストールはなし**
ROS 2 Humble	イメージ内に **apt** で `/opt/ros/humble` に導入済み。`~/.bashrc` が、`/opt/ros/humble` を下地とし、ソースからビルドした `~/airobot_ws` を上乗せする順に `source` して有効にする
画面	コンテナ内のVNCデスクトップを、noVNC を使ってブラウザの http://localhost:6080 で表示
シミュレーション	`~/airobot_ws` の `turtlebot3_gazebo`、ワールド `turtlebot3_world`、ロボット TurtleBot3 Burger

```
docker run -d --name pai_ros2 -p 6080:80 --shm-size=512m -v "$(pwd)":/workspace ¥
  airobotbook/ros2-desktop-ai-robot-book-humble:latest          # コンテナ作成。初回のみ
git clone https://github.com/2life2021/tb3-ai-navigator.git     # コンテナ内の /workspace で
export TURTLEBOT3_MODEL=burger                                    # 必須。.bashrc の既定は happy_mini
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py         # noVNC のターミナルで
python3 scripts/ai_driver.py --model models/policy_v2_aug.pt --reset   # AI自律走行。teleopは停止しておく
python3 scripts/ai_monitor.py                                          # AI視点モニター
```

詳しい手順・確認方法・よくある問題は `docs/SETUP.md`、データ収集・学習・一括評価の手順は README、各モデルを再学習するコマンドは `docs/REPRODUCIBILITY.md` を参照してください。